Integración de Sistemas

Semana 11. Seguridad en integraciones

Sistemas de Información — PUCE
Docente: Damián Nicolalde

Semana 11. Seguridad en integraciones

Objetivo: Comprender los mecanismos fundamentales de autenticación y autorización en sistemas distribuidos (OAuth2, JWT, mTLS), aplicar control de acceso basado en roles y atributos, entender la evolución del modelo de seguridad perimetral hacia Zero Trust, dominar los fundamentos de TLS y buenas prácticas criptográficas, y proteger la mensajería asíncrona con auditoría rigurosa de eventos de seguridad.

Resultado de aprendizaje asociado: Implementar un ecosistema de microservicios expuestos mediante APIs REST y gestionados con API Gateway y un bus de eventos, incorporando contratos, pruebas, observabilidad y seguridad para asegurar escalabilidad y desempeño.

Nota de continuidad: esta semana no tiene contenido previo en contenido/ — se construyó desde el sílabo oficial, igual que las Semanas 5, 7, 8, 9 y 10. Conecta con la Semana 8 (JWT validado en el borde del API Gateway Kong), la Semana 6 (correlation-id y logs estructurados, ahora aplicados a auditoría de seguridad) y la Semana 10 (un intento de autenticación fallido también deja rastro observable).

Introducción

A lo largo de diez semanas has construido un ecosistema funcional: una API REST de Productos, un servicio gRPC, un servidor GraphQL, una integración SOA con mensajería, una Saga de cuatro microservicios comunicados por eventos, contratos formales con JSON Schema, un API Gateway que centraliza el enrutamiento, una disciplina de pruebas automatizadas y contract testing, y observabilidad real con trazas distribuidas. En ningún momento, hasta ahora, alguien ha tenido que demostrar quién es para usar cualquiera de estos servicios. Esto ha sido una simplificación deliberada para concentrarse, semana a semana, en un concepto arquitectónico a la vez — pero es completamente inviable fuera de un aula.

La seguridad en integración de sistemas no es una capa que se agrega al final, como una mano de pintura. Es una propiedad transversal que debe decidirse en cada punto donde dos componentes se comunican: ¿quién puede llamar a este endpoint? ¿qué puede hacer una vez autenticado? ¿puede un tercero interceptar o modificar el mensaje en tránsito? ¿queda un rastro auditable si algo sale mal? Esta semana responde estas cuatro preguntas en orden: quién eres y cómo lo demuestras (11.1), qué puedes hacer una vez demostrado (11.2), cómo cambia el modelo de defensa cuando ya no existe un perímetro claro (11.3), cómo se protege la comunicación en tránsito (11.4), y cómo se extiende todo esto a la mensajería asíncrona, con auditoría como última línea de defensa (11.5).

Vale la pena situar esta semana en la secuencia del curso. La Semana 9 (pruebas y contract testing) dio confianza sobre el comportamiento correcto del sistema. La Semana 10 (observabilidad) dio la capacidad de ver qué está pasando realmente en producción. La seguridad de esta semana depende de ambas: no puedes verificar que un mecanismo de autorización funciona sin pruebas automatizadas que lo confirmen, y no puedes detectar un ataque en curso sin la misma disciplina de logs estructurados y trazas que la Semana 10

desarrolló con una motivación de confiabilidad operativa. Autenticación, autorización, observabilidad y pruebas no son disciplinas separadas — son una sola disciplina de ingeniería de sistemas confiables, vista desde cuatro ángulos distintos.

11.1. OAuth2, JWT, mTLS y scopes

Autenticación en un mundo sin perímetro

En un sistema monolítico clásico, autenticar a un usuario una vez al iniciar sesión bastaba: el servidor mantenía el estado de la sesión en memoria o en una base de datos compartida. En un ecosistema de microservicios, este modelo se rompe: si nestjs-productos-api mantiene sesiones en memoria y el API Gateway de la Semana 8 balancea las peticiones entre varias instancias, una petición puede llegar a una instancia que nunca vio el login original. La solución que se ha impuesto es reemplazar el estado de sesión centralizado por un token autocontenido: una credencial que el cliente presenta en cada solicitud y que cualquier instancia de cualquier servicio puede validar sin consultar una base de datos central de sesiones. JWT es, con enorme diferencia, la forma más extendida de implementar esta idea hoy.

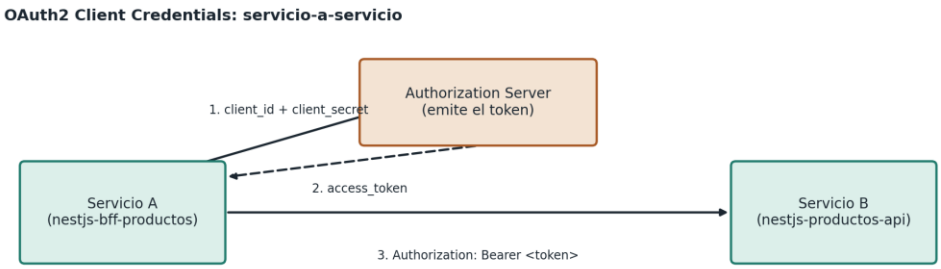
OAuth2: un protocolo de autorización delegada, no de autenticación

Es un error extendido llamar a OAuth2 "el protocolo de login". OAuth2 es, estrictamente, un protocolo de autorización delegada: permite que una aplicación obtenga acceso limitado a un recurso, en nombre de un usuario o de sí misma, sin necesitar la contraseña del usuario. Define varios flujos (grant types), cada uno pensado para un escenario distinto.

Flujo (grant type)	Escenario típico	Quién obtiene el token
Authorization Code	Usuario final inicia sesión en una app web/móvil vía navegador	La app, en nombre del usuario, tras su consentimiento
Client Credentials	Comunicación servicio-a-servicio, sin usuario humano de por medio	El propio servicio, en su propio nombre
Refresh Token	Renovar un access token expirado sin pedir credenciales de nuevo	La app que ya tenía un refresh token válido
Implicit (obsoleto)	Apps de una sola página, sin backend — ya no se recomienda	La app directamente, sin paso intermedio seguro

Para el ecosistema de microservicios de este curso, el flujo relevante es Client Credentials: cuando nestjs-bff-productos de la Semana 8 necesita llamar a nestjs-productos-api, no hay un usuario humano en medio de esa llamada — es un servicio autenticándose ante otro servicio, usando sus propias credenciales (client_id y client_secret), para obtener un token que demuestra su identidad de servicio.

Diagrama 1. Flujo OAuth2 Client Credentials: un servicio solicita un token al Authorization Server y lo usa para llamar a otro servicio.



Anatomía de un JWT

Un JSON Web Token (JWT, RFC 7519) es una cadena de texto con tres partes separadas por puntos, cada una codificada en Base64URL: header, payload y signature.

```
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIxMjMiLCJyb2wiOiJhZG1pbiJ9.4Adcj3UFYzPUVaVF43FmMab6RlaQD8A9V8wFzzht-KQ
HEADER                  PAYLOAD                                  SIGNATURE
```

- Header: metadatos del token, típicamente {"alg": "HS256", "typ": "JWT"} — el algoritmo de firma usado.
- Payload: los claims (afirmaciones) sobre el sujeto. Claims estándar: sub (identificador), exp (expiración), iat (issued at), iss (issuer), aud (audience). Más claims personalizados como rol.
- Signature: firma criptográfica sobre header+payload, con clave secreta (HMAC, simétrico) o par de claves (RSA/ECDSA, asimétrico).

Un detalle crítico y fuente de errores de seguridad reales: el payload de un JWT no está cifrado, solo codificado. Cualquiera que intercepte el token puede leer su contenido con un simple atob(). La firma garantiza que el contenido no fue alterado sin ser detectado — no garantiza confidencialidad. Por eso nunca debe incluirse información sensible (contraseñas, tarjetas, datos médicos) en el payload de un JWT.

Diagrama 2. Estructura de un JWT: header, payload y signature, codificados en Base64URL — no cifrados.



HMAC vs RSA: simétrico vs asimétrico

Cuando un único servicio emite y valida sus propios tokens (como en la práctica de esta semana), HMAC es suficiente y más simple. Cuando varios servicios distintos necesitan validar tokens sin tener autoridad para emitirlos, RSA es la elección correcta: el servicio de identidad guarda la clave privada y firma; todos los demás distribuyen y usan solo la clave pública para verificar, sin poder falsificar un token ellos mismos.

Scopes: acotar el permiso de un token

Un scope es una cadena que declara qué permiso específico otorga un token, más allá de la mera identidad. Un token puede tener el scope productos:read sin productos:write, permitiendo consultar pero no modificar. Los scopes son ortogonales a los roles: limitan lo que ESE token en particular puede hacer, independientemente de lo que el usuario, en general, tendría permitido.

mTLS: confianza mutua sin tokens

TLS mutuo (mTLS) invierte la dirección habitual de la confianza: en TLS estándar solo el servidor presenta un certificado; en mTLS ambas partes presentan y verifican certificados mutuamente. Esto es especialmente valioso en comunicación servicio-a-servicio dentro de una red interna (el escenario típico de un service mesh, Semana 8): la identidad del servicio queda establecida a nivel de la conexión TLS misma. mTLS y JWT no son excluyentes — muchas arquitecturas usan mTLS para la confianza servicio-a-servicio de bajo nivel, y JWT encima para propagar la identidad del usuario final.

Ejercicio resuelto

Enunciado: un compañero propone incluir el email y la contraseña en texto plano dentro del payload de un JWT "para no tener que consultar la base de datos en cada petición". Evalúa la propuesta.

Solución: es un error grave. El payload de un JWT no está cifrado — cualquiera que intercepte el token puede leerlo directamente. Además, la contraseña no debería viajar NUNCA después del login inicial — el JWT existe precisamente para evitar reenviar credenciales. Lo correcto: incluir solo el identificador (sub) y, si el caso lo requiere, el rol — nunca credenciales ni datos sensibles.

Segundo ejercicio resuelto: elegir el flujo OAuth2 correcto

Enunciado: un equipo está diseñando dos integraciones nuevas: (a) permitir que un usuario inicie sesión en la app móvil del curso con su cuenta institucional de Google; (b) permitir que el servicio de Notificaciones llame al servicio de Pagos para consultar el estado de una transacción, sin ningún usuario humano involucrado en ese momento. ¿Qué flujo de OAuth2 corresponde a cada caso?

Solución: el caso (a) es Authorization Code: hay un usuario final que debe iniciar sesión explícitamente y otorgar consentimiento a través de un navegador o vista embebida, y la app recibe el token en su nombre. El caso (b) es Client Credentials: no hay usuario humano de por medio, el servicio de Notificaciones se autentica con sus propias credenciales de servicio (client_id/client_secret) para obtener un token que demuestra su propia identidad, no la de ningún usuario. Confundir estos dos flujos es un error de diseño común: usar Authorization Code para comunicación servicio-a-servicio introduce una complejidad de redirección de navegador completamente innecesaria; usar Client Credentials para un login de usuario final no captura el consentimiento explícito que un login real requiere.

Taller en grupo — Auditar un JWT con IA

Modalidad: parejas, 20 minutos, Nivel de IA: 2 (borrador).

Prompt sugerido: "Aquí está el payload decodificado de un JWT de nuestro sistema: [pega el JSON]. Revisa si contiene información que no debería estar ahí, si el tiempo de expiración es razonable, y si faltan claims estándar recomendables."

Verificación obligatoria: confirmen manualmente que ningún dato sensible real aparece en el payload — es una verificación de seguridad, no de estilo.

11.2. Autorización basada en roles (RBAC/ABAC)

Autenticación vs. autorización

Autenticación responde "¿quién eres?"; autorización responde "¿qué puedes hacer, siendo quien eres?". Un JWT válido demuestra autenticación exitosa — pero un JWT válido de un usuario sin permiso de escritura debe, aun así, ser rechazado al intentar POST /productos. Muchos incidentes de seguridad documentados no son fallos de autenticación (el atacante SÍ tenía una cuenta legítima) sino fallos de autorización (esa cuenta pudo hacer algo que no debería haber podido hacer).

RBAC: Role-Based Access Control

RBAC asigna permisos a roles predefinidos, y a los usuarios se les asigna uno o más roles. En vez de decidir permiso por permiso para cada usuario individual, se decide una sola vez por rol.

Rol	Puede hacer
viewer	GET /productos, GET /productos/:id
editor	Todo lo de viewer + POST /productos, PUT/PATCH /productos/:id
admin	Todo lo de editor + DELETE /productos/:id, gestión de usuarios

RBAC es simple de razonar y de auditar. Su limitación aparece cuando el permiso depende de algo más que el rol fijo: ¿puede un editor modificar CUALQUIER producto, o solo los de su propia sucursal? RBAC puro no expresa bien esa condición sin crear un rol nuevo por cada combinación posible.

Diagrama 3. RBAC vs ABAC: RBAC decide por el rol fijo del usuario; ABAC evalúa una regla sobre atributos del usuario, el recurso y el contexto.



ABAC: Attribute-Based Access Control

ABAC evalúa una regla que combina atributos del sujeto (rol, departamento, ubicación), del recurso (sucursal, categoría) y del contexto (hora del día, dirección IP). Una regla típica: "permitir editar un producto si el usuario tiene rol editor Y el producto pertenece a la misma sucursal Y la hora está dentro del horario laboral". ABAC es más expresivo que RBAC, pero ese poder tiene un costo: las reglas se vuelven más difíciles de auditar de un vistazo, y motores de reglas dedicados (como Open Policy Agent) suelen ser necesarios para no terminar con condicionales dispersos e inconsistentes.

Criterio	RBAC	ABAC
Modelo mental	Roles fijos con permisos predefinidos	Reglas evaluadas dinámicamente sobre atributos
Auditabilidad	Alta — se lee la tabla de roles	Menor — depende del contexto de cada petición
Expresividad	Limitada a combinaciones de roles	Alta — cualquier combinación de atributos
Costo de implementación	Bajo — guards/decoradores simples	Alto — suele requerir motor de reglas dedicado

Guards y decoradores en NestJS

En NestJS, RBAC se implementa idiomáticamente combinando un decorador personalizado que anota el rol requerido, con un Guard que lo lee en tiempo de ejecución.

```
// roles.decorator.ts
export const Roles = (...roles: string[]) => SetMetadata('roles', roles);

// roles.guard.ts
@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}
```

```
  canActivate(context: ExecutionContext): boolean {
    const required = this.reflector.get<string[]>('roles', context.getHandler());
    if (!required) return true;
    const { user } = context.switchToHttp().getRequest();
    return required.includes(user.rol);
  }
}

// productos.controller.ts
@Post()
@Roles('admin')
@UseGuards(JwtAuthGuard, RolesGuard)
crear(@Body() dto: CrearProductoDto) { /* ... */ }
```

El orden de los guards importa: JwtAuthGuard debe ejecutarse primero (para poblar request.user a partir del JWT) y RolesGuard después (para leer ese user.rol ya poblado).

El principio de menor privilegio

Cada identidad (usuaria o de servicio) debe tener el mínimo conjunto de permisos necesario para cumplir su función, ni uno más. Es tentador, por conveniencia durante el desarrollo, dar rol admin a todo mientras se depura — pero esa conveniencia se convierte en una superficie de ataque innecesaria en producción: si esa credencial se filtra, el daño potencial es proporcional al permiso otorgado, no al permiso realmente necesario.

Ejercicio resuelto

Enunciado: el catálogo de Productos ahora se usa en tres sucursales, y un editor de Quito NO debe poder modificar productos de Guayaquil. ¿RBAC puro resuelve esto de forma limpia?

Solución: no de forma limpia sin crear un rol por sucursal, lo cual se vuelve inmanejable si el número de sucursales crece. La solución correcta es un híbrido: mantener RBAC para viewer/editor/admin (simple, auditable) y agregar una regla ABAC puntual: "permitir editar si rol==editor Y producto.sucursalId==usuario.sucursalId". No hace falta migrar todo el sistema a ABAC completo para resolver un caso puntual.

Segundo ejercicio resuelto: RBAC excesivamente granular

Enunciado: un equipo, preocupado por seguir "buenas prácticas", crea 15 roles distintos para su API de Productos: viewer-precio, viewer-stock, viewer-descripcion, editor-precio, editor-stock... ¿es esto una aplicación correcta de RBAC?

Solución: no — es un antipatrón conocido como "explosión de roles" (role explosion). RBAC gana su valor precisamente de su simplicidad de auditoría: poder responder "¿qué puede hacer un editor?" con una sola mirada a la tabla de roles. Con 15 roles finísimamente granulares, esa ventaja desaparece: gestionar qué combinación de roles tiene cada usuario se vuelve tan complejo como el problema que RBAC pretendía simplificar, y en la práctica el equipo termina, de todas formas, necesitando reglas contextuales adicionales para combinarlos correctamente. La señal de que hace falta granularidad por CAMPO o por CONDICIÓN, y no por rol adicional, es

exactamente el momento de considerar ABAC en su lugar — no fragmentar RBAC hasta el punto de perder su propósito original.

Taller en grupo — Diseñar una matriz RBAC con IA

Modalidad: equipos de 3, 25 minutos, Nivel de IA: 2 (borrador).

Prompt sugerido: "Dado este listado de endpoints de nuestra API de Productos, propone una matriz RBAC con 3 roles razonables y qué endpoint puede usar cada uno, justificando por qué cada restricción tiene sentido de negocio."

Verificación obligatoria: revisen que ningún rol reciba más permiso del estrictamente necesario — rechacen cualquier matriz que dé admin por defecto "para simplificar".

11.3. WAF y Zero Trust API Security

Web Application Firewall: filtrado en el borde

Un WAF inspecciona el tráfico HTTP entrante (típicamente en o antes del API Gateway de la Semana 8) buscando patrones conocidos de ataque, antes de que la petición llegue a la lógica de negocio.

Ataque	Patrón que detecta un WAF
Inyección SQL	Fragmentos de sintaxis SQL (' OR 1=1--) en parámetros de entrada
Cross-Site Scripting (XSS)	Etiquetas <script> o atributos de evento JS en campos de texto
Fuerza bruta de credenciales	Volumen anómalo de intentos de login desde el mismo origen
Escaneo de vulnerabilidades	Patrones de solicitudes secuenciales a rutas de administración

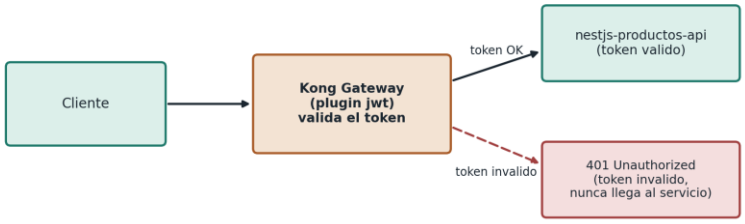
Un WAF es complementario, no sustituto: reduce el volumen de ataques triviales, pero nunca reemplaza la validación y sanitización correcta dentro del propio código.

Del modelo perimetral al modelo Zero Trust

El modelo clásico ("castillo y foso") asume una frontera clara: lo que está dentro de la red es confiable por defecto; lo que viene de fuera pasa por un control estricto en el punto de entrada. Este modelo se rompe con los microservicios modernos: el perímetro ya no es una frontera geográfica clara, y si un atacante compromete UN servicio interno, el modelo perimetral le otorga movimiento lateral casi sin fricción hacia el resto.

Zero Trust invierte la premisa: "nunca confíes, siempre verifica". Cada petición, sin importar si viene de fuera de la red o de un servicio adyacente, se autentica y autoriza de forma explícita e independiente — como ilustra la validación de un JWT en el propio borde del sistema.

Diagrama 6. Zero Trust en la práctica: incluso en el borde, el API Gateway (Kong, Semana 8) valida explícitamente cada token antes de dejar pasar la petición.



Aspecto	Modelo perimetral	Zero Trust
Confianza por defecto	Alta dentro del perímetro	Nula, en cualquier punto
Punto de verificación	Solo en el borde (firewall/VPN)	En cada solicitud, en cada servicio
Riesgo de movimiento lateral	Alto si el perímetro es cruzado una vez	Bajo — cada salto requiere verificación propia
Encaje con microservicios	Pobre — el perímetro ya no es claro	Natural — cada servicio es su propio perímetro

Zero Trust no es un producto, es un principio de diseño

Es un error tratar Zero Trust como un producto que se compra e instala. Es un principio arquitectónico que se materializa en decisiones concretas ya vistas: JWT/mTLS validado en cada servicio, no solo en el borde (11.1); guards de autorización explícitos en cada endpoint sensible (11.2); y control de acceso también sobre colas y tópicos de mensajería (11.5).

Ejercicio resuelto

Enunciado: una empresa argumenta: "nuestra red interna ya está protegida por una VPN, así que no necesitamos autenticar las llamadas entre nuestros propios microservicios". Evalúa el argumento bajo Zero Trust.

Solución: el argumento asume el modelo perimetral clásico. Si un atacante compromete un solo contenedor dentro de esa VPN, la VPN no ofrece ninguna barrera adicional entre ese contenedor comprometido y el resto de los microservicios. La recomendación Zero Trust es autenticar y autorizar cada llamada interna igual que una externa, típicamente con mTLS y/o JWT de servicio-a-servicio, además de la VPN (defensa en profundidad, no sustitución).

Segundo ejercicio resuelto: WAF vs. validación en el código

Enunciado: un equipo, tras instalar un WAF delante de su API, decide eliminar la validación de entrada (sanitización de parámetros) del propio código "porque ya el WAF filtra los ataques comunes". Evalúa esta decisión.

Solución: es una decisión riesgosa que confunde una capa complementaria con una sustituta. Un WAF opera por reconocimiento de patrones conocidos — es efectivo contra ataques comunes y bien documentados, pero por diseño no puede anticipar cada variante posible de un ataque, especialmente payloads ofuscados o técnicas nuevas no incluidas todavía en sus reglas. La validación dentro del propio código (consultas parametrizadas contra inyección SQL, escapado correcto contra XSS) protege independientemente de si el patrón específico del ataque es reconocido por el WAF o no. La práctica correcta es defensa en profundidad: el WAF como primera capa de filtrado de ataques masivos y conocidos, Y la validación rigurosa en el código como garantía de fondo que no depende de que ninguna herramienta externa "adivine" correctamente cada ataque posible.

Taller en grupo — Decidir el nivel de Zero Trust con IA

Modalidad: parejas, 20 minutos, Nivel de IA: 3 (revisor).

Prompt sugerido: "Para estos tres escenarios [blog personal; sistema de pagos con 5 microservicios; hospital con historiales clínicos], evalúa si conviene invertir en Zero Trust completo o si la seguridad perimetral básica es proporcional al riesgo."

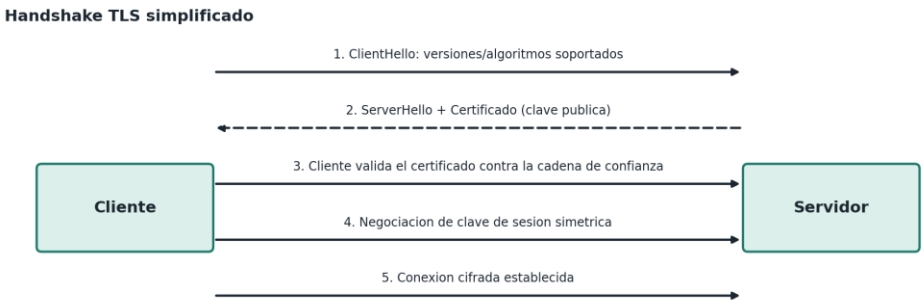
Verificación obligatoria: discutan el argumento contrario para el blog personal — Zero Trust completo también tiene un costo que no siempre está justificado.

11.4. Cifrado TLS y prácticas criptográficas

TLS: el fundamento de la comunicación segura en tránsito

TLS protege una conexión garantizando confidencialidad, integridad y autenticidad del servidor. El handshake simplificado: el cliente propone versiones/algoritmos; el servidor responde con su certificado (clave pública firmada por una CA) y elige algoritmos; el cliente valida el certificado contra la cadena de confianza; ambas partes negocian una clave de sesión simétrica para el resto de la conversación.

Diagrama 4. Handshake TLS simplificado: intercambio de certificado, verificación de la cadena de confianza y negociación de la clave de sesión.



Cadena de confianza y certificados autofirmados

Un certificado es confiable si fue firmado por una Autoridad Certificadora (CA) cuya clave pública ya está preinstalada como confiable en el sistema/navegador. Para desarrollo, es común generar un certificado autofirmado — cifra igual que uno firmado por una CA real, pero el cliente mostrará una advertencia porque no puede verificar la cadena de confianza. En producción se usan certificados de una CA pública (o servicios automatizados como Let's Encrypt).

Cifrado vs. hashing

	Cifrado	**Hashing**
Reversibilidad	Reversible (con la clave correcta)	Irreversible por diseño
Propósito	Proteger datos que necesitas recuperar después	Verificar sin necesitar recuperar el original
Ejemplo correcto	Cifrar un archivo de configuración con secretos	Almacenar el hash de una contraseña
Algoritmos típicos	AES (simétrico), RSA (asimétrico)	bcrypt, argon2 (deliberadamente lentos)

Los algoritmos de hashing para contraseñas (bcrypt, argon2) están diseñados deliberadamente para ser computacionalmente costosos — a diferencia de un hash rápido como SHA-256 (pensado para verificar integridad de archivos), un hash de contraseña lento hace que un ataque de fuerza bruta contra millones de contraseñas filtradas sea impracticablemente lento.

```
// Correcto: usar bcrypt para hashear contraseñas antes de guardarlas
import * as bcrypt from 'bcrypt';
const hash = await bcrypt.hash(passwordEnTextoPlano, 10);
const esValida = await bcrypt.compare(passwordIngresada, hashGuardado);
```

Buenas prácticas criptográficas

1. Nunca implementes tu propio algoritmo criptográfico — usa siempre bibliotecas revisadas y auditadas (bcrypt, crypto nativo de Node).

2. Gestión de secretos: nunca hardcodear claves o contraseñas en el código fuente. Usar variables de entorno como mínimo, y un gestor de secretos dedicado en producción.

3. Rotación de claves: una clave comprometida debe poder reemplazarse sin re-desplegar toda la aplicación desde cero.

4. Nunca loguear secretos: un log de depuración no debe incluir el header Authorization completo ni contraseñas en texto plano.

Ejercicio resuelto

Enunciado: encuentras este código: crypto.createHash('sha256').update(password).digest('hex') usado para guardar contraseñas. ¿Qué está mal?

Solución: SHA-256 es rápido por diseño — la propiedad opuesta a la deseada para contraseñas. Un atacante con la base de hashes filtrada puede probar miles de millones de contraseñas por segundo contra SHA-256, mientras que el mismo ataque contra bcrypt tomaría órdenes de magnitud más tiempo. La corrección: reemplazar por bcrypt.hash() o argon2, nunca un hash de propósito general para contraseñas.

Segundo ejercicio resuelto: rotación de claves sin downtime

Enunciado: una organización descubre que el secreto usado para firmar sus JWT quedó accidentalmente expuesto en un repositorio público durante unas horas. ¿Basta con cambiar el secreto inmediatamente en el servidor de producción?

Solución: cambiar el secreto es necesario pero, por sí solo, incompleto e insuficiente si se hace sin planificación: todos los JWT ya emitidos con el secreto anterior, y que todavía no han expirado, dejarán de validarse instantáneamente en el momento del cambio — cada usuario con sesión activa sería desconectado de golpe (una interrupción de servicio real, aunque autoinfligida). La práctica correcta de rotación de claves sin downtime es tener la capacidad de aceptar temporalmente AMBAS claves (la antigua y la nueva) durante una ventana de transición igual o mayor al tiempo de expiración máximo de un token, para permitir que los tokens antiguos expiren de forma natural mientras los nuevos ya se firman con la clave nueva — solo después de esa ventana se retira por completo la clave comprometida. Este es precisamente el tipo de escenario que motiva diseñar, desde el principio, la infraestructura de firma para soportar más de una clave activa simultáneamente, en vez de asumir ingenuamente una única clave fija de por vida.

Taller en grupo — Revisar código con IA

Modalidad: parejas, 20 minutos, Nivel de IA: 3 (revisor).

Prompt sugerido: "Revisa este fragmento que maneja registro y login [pega un fragmento con contraseña en texto plano, secreto de JWT hardcodeado, sin expiración] y enumera cada mala práctica, ordenadas por severidad."

Verificación obligatoria: confirmen que la IA detectó las tres fallas deliberadas — si omitió alguna, complétenla manualmente.

11.5. Seguridad en colas, mensajería y auditoría de eventos

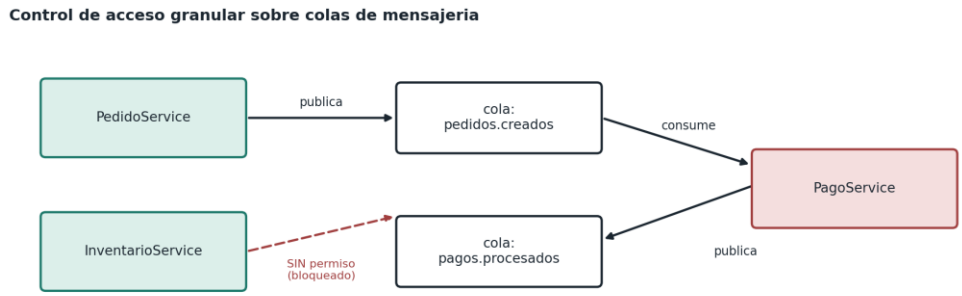
Proteger el broker: TLS y credenciales

La mensajería asíncrona (RabbitMQ/Kafka, Semanas 5 y 6) introduce una superficie de ataque adicional que las guías centradas solo en HTTP suelen pasar por alto: el propio broker. Por defecto, muchas instalaciones de desarrollo de RabbitMQ exponen su puerto sin TLS y con credenciales por defecto (guest/guest) — aceptable para un contenedor local de práctica, pero crítico si llegara así a producción. Medidas mínimas: TLS en las conexiones AMQP, credenciales únicas por servicio, y restricción de red.

Control de acceso a colas y tópicos

Zero Trust (11.3) también aplica a la mensajería: no todo servicio necesita poder publicar en toda cola, ni consumir de todo tópico. RabbitMQ y Kafka permiten definir permisos granulares por usuario, aplicando el mismo principio de menor privilegio de 11.2, ahora a nivel de infraestructura de mensajería.

Diagrama 5. Control de acceso granular sobre colas: cada microservicio tiene permiso solo sobre las colas relevantes a su rol de negocio.



Por qué auditar eventos de seguridad

Autenticación y autorización, por bien implementadas que estén, eventualmente fallarán ante algún escenario no anticipado. La auditoría es la última línea de defensa: no previene el incidente, pero garantiza que exista evidencia suficiente para reconstruir qué pasó, cuándo, y quién lo hizo.

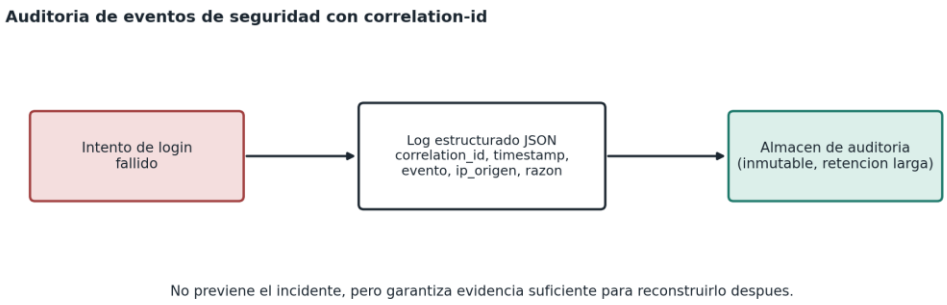
Evento	Por qué importa
Login exitoso y fallido	Detectar patrones de fuerza bruta o acceso desde ubicaciones inusuales
Acceso denegado por rol (403)	Detectar intentos de escalación de privilegios
Cambio de permisos/roles	Reconstruir quién otorgó qué permiso y cuándo
Publicación/consumo inusual en cola sensible	Detectar un servicio comprometido actuando fuera de patrón

Auditoría con logs estructurados y correlation-id

La técnica de logging estructurado con correlation-id, desarrollada en la Semana 6 y llevada a trazas reales en la Semana 10, es exactamente la misma técnica que sostiene una auditoría de seguridad efectiva.

```
{
  "timestamp": "2026-02-14T09:03:11Z",
  "event": "auth.login.failed",
  "correlation_id": "cid-a91f2",
  "usuario_intentado": "admin@example.com",
  "ip_origen": "203.0.113.44",
  "razon": "credenciales_invalidas"
}
```

Diagrama 7. Auditoría de eventos de seguridad: un evento sensible genera un log estructurado inmutable de retención larga.



La diferencia clave de un log de auditoría de seguridad frente a un log operativo genérico: debe ser inmutable (no debe poder alterarse ni borrarse por el mismo sistema que lo generó) y debe retenerse el tiempo suficiente para investigaciones forenses posteriores, que a menudo ocurren semanas o meses después del evento original.

Ejercicio resuelto

Enunciado: un usuario con rol viewer logró eliminar productos. Los logs no registran ningún intento de DELETE de ese usuario en semanas. ¿Qué falta en el diseño de auditoría, independientemente de la causa raíz del bug?

Solución: falta el registro sistemático de TODO acceso a operaciones sensibles, exitoso o denegado, sin depender de que el código de negocio "recuerde" loguear cada caso. Un sistema robusto se implementa como un interceptor/middleware transversal que registra cada intento ANTES de que el guard decida permitirlo o denegarlo — así, incluso si el guard tiene un bug, queda evidencia completa para reconstruir el incidente.

Taller en grupo — Diseñar una política de auditoría con IA

Modalidad: equipos de 3, 25 minutos, Nivel de IA: 2 (borrador).

Prompt sugerido: "Para nuestra API de Productos con JWT+RBAC, propone qué 5 eventos de seguridad deberíamos auditar como mínimo, qué campos debería tener cada log, y durante cuánto tiempo retenerlos."

Verificación obligatoria: verifiquen que la propuesta incluya explícitamente los intentos DENEGADOS (403/401), no solo los exitosos.

OWASP API Security Top 10 aplicado al curso

El OWASP API Security Top 10 es la referencia más citada de la industria para clasificar vulnerabilidades específicas de APIs (distinta del OWASP Top 10 general de aplicaciones web). Repasar sus categorías, mapeadas explícitamente a lo que ya construiste en este curso, sirve como una auditoría de cierre de todo el ecosistema de Productos:

Categoría OWASP	Riesgo	Dónde se cubre en el curso
API1: Broken Object Level Authorization	Un usuario accede a datos de otro usuario cambiando un ID en la URL	RBAC/ABAC (11.2) — verificar propiedad del recurso, no solo el rol
API2: Broken Authentication	Mecanismos de login débiles o mal implementados	JWT + bcrypt (11.1, 11.4) — tokens firmados, contraseñas hasheadas correctamente
API3: Broken Object Property Level Authorization	Un usuario modifica campos que no debería poder cambiar (ej. su propio rol)	DTOs explícitos (Semana 2) que excluyen campos sensibles del payload aceptado
API4: Unrestricted Resource Consumption	Un cliente satura la API con volumen excesivo de solicitudes	Rate-limiting en el API Gateway (Semana 8)
API5: Broken Function Level Authorization	Un usuario invoca una operación administrativa sin tener el rol adecuado	RolesGuard con @Roles() (11.2)
API6: Unrestricted Access to Sensitive Business Flows	Automatización abusiva de un flujo de negocio legítimo (ej. compra masiva por bots)	Rate-limiting + auditoría de patrones anómalos (11.5)
API7: Server Side Request Forgery	El servidor es engañado para hacer una petición a un destino no confiable	Validación estricta de URLs/entradas externas (fuera del alcance directo de esta semana, mencionado como referencia)
API8: Security Misconfiguration	Configuración insegura por defecto (credenciales guest/guest, CORS abierto)	Credenciales propias del broker (11.5), gestión de secretos (11.4)
API9: Improper Inventory Management	APIs "fantasma" o versiones viejas sin documentar ni proteger, olvidadas en producción	Documentación de contratos (Semana 7) y gobernanza de rutas en el Gateway (Semana 8)
API10: Unsafe Consumption of APIs	Confiar ciegamente en datos de una API de terceros sin validarlos	Validación de esquemas también en las respuestas de servicios externos consumidos (Semana 7)

Un hallazgo notable de este mapeo: ninguna categoría del Top 10 se resuelve con una sola herramienta o una sola semana del curso. Cada una es, en realidad, la ausencia o falla de una combinación de las prácticas ya vistas — evidencia de que la seguridad de una API real es sistémica, no una casilla aislada que se marca una sola vez.

Caso de estudio: reconstrucción de un incidente de seguridad

Miércoles, 3:47 a.m. Un sistema de alertas automatizado (configurado siguiendo la disciplina de la Semana 10) dispara una notificación: la tasa de respuestas 401 en nestjs-productos-api subió de un valor basal de menos de 1 por minuto a más de 200 por minuto, sostenido durante los últimos 8 minutos.

3:52 a.m. El ingeniero de guardia abre el dashboard de Grafana y confirma el patrón: todas las respuestas 401 provienen de un rango reducido de direcciones IP, y el intervalo entre solicitudes sucesivas desde cada IP es extremadamente regular — un patrón característico de un script automatizado, no de tráfico humano orgánico.

3:58 a.m. Busca en Kibana (ELK, Semana 10) los logs de auditoría correspondientes a esa ventana de tiempo, filtrando por event: auth.login.failed. Encuentra miles de intentos de login con el mismo campo usuario_intentado (admin@empresa.com) y contraseñas distintas en cada intento — la firma inequívoca de un ataque de fuerza bruta de credenciales dirigido específicamente a una cuenta administrativa conocida.

4:05 a.m. Revisa si alguno de esos miles de intentos tuvo éxito. Ninguno — el hashing con bcrypt (11.4) hizo que cada intento de verificación de contraseña tomara el tiempo suficiente para que el volumen de intentos posible en 8 minutos fuera insuficiente para agotar el espacio de contraseñas razonables, y ninguna de las contraseñas probadas coincidió con el hash real almacenado.

4:10 a.m. Aun sin una brecha exitosa, el ingeniero toma dos acciones: primero, bloquea temporalmente el rango de IPs de origen a nivel del API Gateway (Kong, Semana 8) usando el plugin de rate-limiting ya configurado en la práctica de esa semana, ahora con un límite más agresivo específicamente para el endpoint /auth/login; segundo, documenta el incidente completo en el sistema de auditoría, referenciando el correlation_id del primer intento detectado, para que quede como evidencia trazable si el mismo patrón de ataque reaparece semanas después desde otro rango de IPs.

4:20 a.m. Como medida preventiva de más largo plazo, propone en el reporte post-incidente agregar un mecanismo de bloqueo progresivo de cuenta (account lockout) tras N intentos fallidos consecutivos en un periodo corto, complementario al rate-limiting ya existente en el gateway — el rate-limiting protege la infraestructura de un volumen alto de solicitudes, pero el bloqueo de cuenta protege específicamente la cuenta objetivo del ataque, incluso si el atacante distribuye sus intentos entre muchas IPs distintas para evadir el límite por IP.

Este incidente, resuelto sin ninguna brecha real, ilustra cómo las cinco piezas de esta semana operan juntas en la práctica: el hashing lento con bcrypt (11.4) hizo la fuerza bruta impracticable en el tiempo disponible; la auditoría con correlation-id (11.5) permitió confirmar en minutos que ningún intento tuvo éxito; y el rate-limiting del API Gateway (11.3, construido en la Semana 8) proporcionó el mecanismo de contención inmediata. Ninguna de las tres piezas por sí sola habría bastado: sin hashing lento, la fuerza bruta podría haber tenido éxito; sin auditoría, el

ingeniero no habría podido confirmarlo con certeza; sin rate-limiting, el ataque habría continuado consumiendo recursos indefinidamente incluso sin éxito.

Ejercicio integrador de cierre

Enunciado: retoma el incidente anterior. Supón ahora una variante: el atacante, en vez de probar miles de contraseñas contra la cuenta admin@empresa.com, obtuvo por otro medio (una filtración externa no relacionada con tu sistema) un JWT válido y no expirado que pertenece legítimamente a un usuario con rol viewer, y lo usa para intentar repetidamente DELETE /productos/:id. Responde en orden: (a) ¿el mecanismo de bcrypt del caso anterior ofrece alguna protección en este nuevo escenario? (b) ¿qué capa SÍ detiene esta operación, y en qué sección de esta semana se explica? (c) ¿qué evento debería quedar auditado como resultado? (d) si el mismo token se sigue usando de forma insistente durante días, ¿qué acción adicional, más allá de que cada intento individual sea rechazado, sería razonable tomar?

Solución: (a) No, ninguna: bcrypt protege específicamente el proceso de verificación de contraseñas durante el login; una vez que el atacante ya posee un JWT válido, el paso de autenticación por contraseña quedó completamente evitado — el JWT ES la credencial en este punto, y su robo hace irrelevante qué tan robusto haya sido el hashing original. (b) La capa de autorización (RBAC, sección 11.2): el JWT es válido y demuestra autenticación exitosa (el token no fue falsificado, fue robado), pero el rol viewer codificado en su payload no incluye permiso para DELETE — el RolesGuard rechaza la operación con 403, independientemente de que el token en sí sea auténtico. Este es exactamente el caso que motiva la distinción de 11.2 entre autenticación (quién eres, y en este caso el token SÍ demuestra correctamente quién es el usuario robado) y autorización (qué puedes hacer, y ese usuario nunca pudo hacer DELETE, con o sin robo de token). (c) Cada uno de esos intentos debería auditarse como un evento de acceso denegado por rol (403), con su correlation_id, el sub del JWT usado y el timestamp — exactamente el tipo de evento descrito en la tabla de 11.5. (d) Un volumen alto y sostenido de 403 desde el mismo sub, incluso sin ningún éxito, es en sí mismo una señal de alerta que amerita investigar si esa cuenta fue comprometida (el usuario legítimo probablemente no está intentando repetidamente una operación que sabe que no tiene permitida) y considerar revocar manualmente ese token específico antes de su expiración natural — lo cual exige que el sistema tenga, desde el diseño, algún mecanismo de revocación (una lista de tokens invalidados, o tokens de vida más corta con refresh) más allá de simplemente esperar a que el exp del JWT se cumpla por sí solo.

Autoevaluación: preguntas de repaso

1. ¿Por qué OAuth2 se describe como un protocolo de autorización y no de autenticación?

Porque su propósito central es otorgar acceso limitado a un recurso en nombre de alguien, no verificar una identidad.

2. ¿Por qué nunca se debe incluir una contraseña en el payload de un JWT?

Porque el payload está codificado en Base64URL, no cifrado — cualquiera que intercepte el token puede leerlo.

3. ¿Cuándo conviene usar RSA en vez de HMAC para firmar JWT?

Cuando varios servicios distintos necesitan validar tokens sin tener autoridad para emitirlos.

4. ¿Qué diferencia fundamental hay entre RBAC y ABAC?

RBAC asigna permisos a roles fijos predefinidos; ABAC evalúa reglas dinámicas sobre atributos del usuario, recurso y contexto.

5. ¿Por qué Zero Trust es especialmente relevante en microservicios?

Porque el perímetro de red ya no es una frontera clara, y el modelo perimetral permite movimiento lateral sin fricción si un servicio es comprometido.

6. ¿Qué garantiza TLS que un cifrado simétrico simple no garantizaría?

La autenticidad del servidor: que el cliente se conecta al servidor legítimo, verificado contra una cadena de confianza.

7. ¿Por qué bcrypt es preferible a SHA-256 para hashear contraseñas?

Porque bcrypt es deliberadamente lento, haciendo impracticable la fuerza bruta a gran escala; SHA-256 es rápido por diseño.

8. ¿Por qué un broker con credenciales por defecto es un riesgo real?

Porque cualquiera que las conozca puede publicar o consumir mensajes arbitrarios si el broker es accesible desde fuera de la red de confianza.

9. ¿Por qué la auditoría no debe depender del código de negocio?

Porque si el bug que causó el incidente está en esa misma lógica, es probable que también omita el log — debe ser una capa transversal independiente.

10. ¿Qué principio comparten RBAC, ABAC y el control de acceso a colas?

El principio de menor privilegio: cada identidad debe tener el mínimo permiso necesario para su función.

Checklist de higiene de seguridad previa a producción

Antes de considerar que cualquier servicio de este curso (o de tu carrera profesional) está listo para exponerse fuera de un entorno de práctica controlado, esta lista condensa las verificaciones mínimas que sintetizan las cinco secciones de la semana:

- Ningún secreto (clave de JWT, contraseña de base de datos, credencial de broker) está hardcodeado en el código fuente ni en el historial de Git.
- Toda contraseña de usuario se almacena con bcrypt o argon2 — nunca en texto plano, nunca con un hash de propósito general como SHA-256 o MD5.
- Cada endpoint sensible tiene tanto autenticación (¿quién eres?) como autorización explícita (¿qué puedes hacer?) — ninguna de las dos se asume por la presencia de la otra.
- El tiempo de expiración de los tokens de acceso es corto (minutos, no días), con un mecanismo de refresh separado si se necesita sesión prolongada.
- Las conexiones a bases de datos y brokers de mensajería usan credenciales propias por servicio, nunca una credencial compartida tipo admin/admin o guest/guest.
- Todo tráfico en tránsito, tanto externo (cliente-API) como interno (servicio-a-servicio), usa TLS — no solo el tráfico que cruza la frontera de la red corporativa.
- Existe auditoría activa de al menos: logins fallidos, accesos denegados por rol, y cambios de permisos — con retención suficiente para una investigación forense posterior al incidente.
- El principio de menor privilegio se aplicó explícitamente a cada rol, credencial de servicio y permiso de cola de mensajería — nadie tiene más acceso del estrictamente necesario para su función.

Esta checklist no sustituye una auditoría de seguridad profesional completa antes de un despliegue real con datos de usuarios reales, pero cubre la enorme mayoría de los errores más comunes y más costosos documentados de forma recurrente en incidentes de seguridad de la industria — la mayoría de las brechas reales no explotan vulnerabilidades sofisticadas de día cero, sino exactamente estos descuidos básicos y evitables.

Tabla resumen de la semana

Concepto	En una frase
OAuth2	Protocolo de autorización delegada; Client Credentials es el flujo relevante para servicio-a-servicio
JWT	Token autocontenido, codificado (no cifrado), firmado con HMAC o RSA
mTLS	Autenticación mutua a nivel de conexión TLS, sin depender de tokens
RBAC	Permisos asignados a roles fijos, simple de auditar

ABAC	Permisos evaluados por reglas dinámicas sobre atributos
Zero Trust	Verificar cada petición sin importar su origen
TLS	Confidencialidad, integridad y autenticidad del servidor en una conexión
Hashing de contraseñas	Irreversible y deliberadamente lento (bcrypt/argon2)
Auditoría de eventos	Última línea de defensa: no previene, pero permite reconstruir un incidente

Conclusión de la semana

La seguridad, como la observabilidad de la Semana 10, no es una casilla que se marca instalando una librería — es una disciplina que se construye decisión a decisión: qué mecanismo de identidad usar y por qué (11.1), qué puede hacer cada identidad una vez verificada (11.2), cuánta confianza implícita se otorga por el simple hecho de estar "dentro" de la red (11.3), cómo se protege la comunicación en tránsito (11.4), y qué evidencia queda cuando, inevitablemente, algo falla (11.5). Estas cinco piezas no son compartimentos aislados: el caso de estudio de esta semana mostró cómo el hashing lento, el rate-limiting del gateway y la auditoría con correlation-id tuvieron que operar juntas para contener un solo incidente, y el ejercicio integrador demostró que un token robado (fallo de confidencialidad) sigue estando acotado por la autorización basada en roles (una capa completamente distinta) — ninguna defensa individual es suficiente por sí sola, y esa es precisamente la lógica detrás del principio de defensa en profundidad que atraviesa toda la semana.

El ecosistema de Productos que has construido desde la Semana 1 termina esta semana con una propiedad que no tenía al principio: ya no es un sistema ingenuamente abierto. Cada llamada relevante ahora demuestra una identidad, esa identidad tiene un permiso explícito y verificado, la comunicación viaja protegida, y cualquier intento de abuso queda registrado con suficiente detalle para investigarlo. Esta transformación —de un prototipo funcional a un sistema con una postura de seguridad razonada— es exactamente el tipo de madurez que se espera de cualquier sistema antes de exponerlo a usuarios reales, y es la última pieza de fundamentos técnicos antes de que el curso se dirija, en las semanas siguientes, hacia la interoperabilidad entre dominios heterogéneos y la resiliencia operativa del ecosistema completo.

Preguntas frecuentes

¿Necesito implementar OAuth2 completo con un Authorization Server dedicado para la práctica de esta semana? No. La práctica se centra en JWT emitido directamente por tu propio nestjs-productos-api (un endpoint /auth/login que firma el token), que es suficiente para entender los conceptos de autenticación y autorización. Un Authorization Server dedicado (como Auth0, Keycloak o un servidor OAuth2 propio) es una evolución posterior, típica de sistemas con múltiples aplicaciones cliente, fuera del alcance de esta semana.

¿Es obligatorio implementar mTLS en la práctica? No. mTLS se presenta conceptualmente en 11.1 y 11.3 porque es relevante para entender las alternativas de confianza servicio-a-servicio, pero la práctica usa JWT como mecanismo principal, más accesible para una primera implementación en una sesión de laboratorio.

¿Qué pasa si mi contraseña hasheada con bcrypt se ve distinta cada vez que hasheo la misma contraseña? Es el comportamiento esperado y correcto: bcrypt incorpora automáticamente un valor aleatorio (salt) distinto en cada llamada, por lo que el mismo texto plano produce hashes distintos cada vez — esto es una protección adicional contra ataques de tabla precalculada (rainbow tables). bcrypt.compare() sabe extraer el salt del hash almacenado para verificar correctamente sin que tengas que gestionarlo tú mismo.

Anexo: comandos y configuración de referencia rápida

Este anexo reúne los comandos y fragmentos de configuración que usarás en la práctica de esta semana — referencia rápida durante el laboratorio, sin releer toda la teoría.

Dependencias para JWT + RBAC en NestJS

```
npm install @nestjs/jwt @nestjs/passport passport passport-jwt bcrypt
npm install -D @types/passport-jwt @types/bcrypt
```

Generar el hash de una contraseña de prueba (script rápido)

```
const bcrypt = require('bcrypt');
bcrypt.hash('mi-contrasena-de-prueba', 10).then(console.log);
```

Estructura mínima de un guard JWT

```
// jwt.strategy.ts
@Injectable()
export class JwtStrategy extends PassportStrategy(Strategy) {
  constructor() {
    super({
      jwtFromRequest: ExtractJwt.fromAuthHeaderAsBearerToken(),
      secretOrKey: process.env.JWT_SECRET,
    });
  }
  async validate(payload: any) {
    return { userId: payload.sub, rol: payload.rol };
  }
}
```

Generar certificados TLS autofirmados con openssl

```
openssl req -nodes -new -x509 -keyout server.key -out server.cert -days 365
# Genera server.key (clave privada) y server.cert (certificado autofirmado)
```

Errores comunes al implementar JWT+RBAC por primera vez

- Guardar el secreto de JWT directamente en el código en vez de una variable de entorno — revisar antes de cada commit.

- Invertir el orden de los guards (RolesGuard antes que JwtAuthGuard): RolesGuard intenta leer request.user antes de que exista.
- Definir un tiempo de expiración (exp) demasiado largo para el access token, aumentando la ventana de riesgo si el token se filtra.
- Omitir la verificación de expiración al decodificar manualmente un JWT sin usar la librería de validación completa — decodificar no es lo mismo que validar.
- No manejar el caso de un JWT bien formado pero con un rol que ya no existe en el sistema (un usuario cuyo rol fue revocado después de emitido el token).

Checklist de verificación de la práctica

- POST /auth/login devuelve un JWT válido con el rol correcto en el payload.
- GET /productos sin token devuelve 401; con token válido devuelve 200.
- POST /productos con un token de rol viewer devuelve 403; con rol admin devuelve 201.
- Cada intento de login (exitoso o fallido) y cada 403 quedan registrados en un log estructurado con correlation-id.
- El plugin jwt de Kong (si se completó el reto) rechaza un token inválido antes de que la petición llegue al microservicio.

Glosario de términos

- OAuth2: protocolo de autorización delegada que permite obtener acceso limitado a un recurso sin necesitar las credenciales del propietario.
- JWT (JSON Web Token): token autocontenido de tres partes (header, payload, signature) usado para propagar identidad y claims de forma verificable.
- mTLS: TLS mutuo, donde cliente y servidor presentan y verifican certificados.
- Scope: permiso específico y acotado otorgado a un token, independiente del rol general del usuario.
- RBAC: modelo de autorización basado en roles fijos predefinidos.
- ABAC: modelo de autorización basado en reglas evaluadas sobre atributos del sujeto, recurso y contexto.
- WAF: capa de filtrado que detecta y bloquea patrones de ataque HTTP conocidos en el borde del sistema.
- Zero Trust: principio de seguridad que exige verificar cada petición explícitamente, sin confianza implícita por origen de red.
- TLS: protocolo que garantiza confidencialidad, integridad y autenticidad del servidor en una conexión de red.
- Hashing: transformación irreversible usada para verificar datos sin necesitar recuperarlos.
- Principio de menor privilegio: cada identidad debe tener el mínimo permiso necesario para su función.
- Auditoría de eventos de seguridad: registro inmutable y duradero de eventos sensibles para investigación forense posterior.

Referencias

Hardt, D. (2012). The OAuth 2.0 Authorization Framework. RFC 6749. https://www.rfc-editor.org/rfc/rfc6749

Jones, M., Bradley, J., & Sakimura, N. (2015). JSON Web Token (JWT). RFC 7519. https://www.rfc-editor.org/rfc/rfc7519

OWASP Foundation. (2023). OWASP API Security Top 10. https://owasp.org/API-Security/

NIST. (2020). Zero Trust Architecture (SP 800-207). https://csrc.nist.gov/publications/detail/sp/800-207/final

Passport.js. (2024). Passport.js Documentation. https://www.passportjs.org/

RabbitMQ. (2024). Access Control Guide. https://www.rabbitmq.com/access-control.html